Digital Electronics Lab

Truth Tables and Functions of 50 Circuits

Conventions: 1 = HIGH, 0 = LOW, X = don't care / invalid. Q_n = present state, Q_{n+1} = next state. A' means NOT A, · means AND, + means OR, ⊕ means XOR. In every table the grey header columns are inputs and the blue columns are outputs.

Note: The lab index lists the Boolean-expression experiments (3-, 4- and 2-variable) without giving the expressions. Example expressions are used here (F = AB + A'C, F = AB + C'D, F = A + B'); substitute the expressions from your lab manual. Flip-flop tables assume the clock is enabled (CLK = 1).

1. 1×4 Demultiplexer

Function: Routes the single data input D to one of four outputs, chosen by select lines S1 S0. Y0 = D·S1'·S0', Y1 = D·S1'·S0, Y2 = D·S1·S0', Y3 = D·S1·S0. Unselected outputs stay 0.

D	S1	S0	Y0	Y1	Y2	Y3
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

2. 2×4 Decoder

Function: Converts a 2-bit binary input into a one-hot output (exactly one of 4 lines is HIGH). D0 = A'B', D1 = A'B, D2 = AB', D3 = AB.

A	B	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

3. 3×8 Decoder

Function: Converts a 3-bit binary input into a one-hot output on one of 8 lines. Each output Dk is the minterm m_k of A B C (e.g. D5 = A·B'·C).

A	B	C	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

4. 4×1 Multiplexer

Function: Selects one of four data inputs I0–I3 and passes it to the single output Y, under control of S1 S0. Y = S1'S0'·I0 + S1'S0·I1 + S1S0'·I2 + S1S0·I3.

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

5. 8×1 Multiplexer

Function: Selects one of eight data inputs I0–I7 and passes it to the output Y, under control of S2 S1 S0. Y = $\Sigma m_k(S2,S1,S0)\cdot I_k$.

S2	S1	S0	Y
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3
1	0	0	I4
1	0	1	I5
1	1	0	I6
1	1	1	I7

6. Octal to Binary Encoder (8 : 3)

Function: Encodes the one active input line D0–D7 into a 3-bit binary code. A2 = D4+D5+D6+D7, A1 = D2+D3+D6+D7, A0 = D1+D3+D5+D7. (Only one input active at a time.)

D0	D1	D2	D3	D4	D5	D6	D7	A2	A1	A0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

7. Decimal to BCD Encoder (10 : 4)

Function: Encodes the one active decimal input D0–D9 into a 4-bit BCD code. A3 = D8+D9, A2 = D4+D5+D6+D7, A1 = D2+D3+D6+D7, A0 = D1+D3+D5+D7+D9.

D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	A3	A2	A1	A0
1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0	0	0	1	0	1
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	0	0	0	0	0	0	1	0	0	0	1	1	1
0	0	0	0	0	0	0	0	1	0	1	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	1

8. 2×1 Multiplexer

Function: Selects one of two inputs I0, I1 using select line S. Y = S'·I0 + S·I1 (2 AND, 1 OR, 1 NOT).

S	I0	I1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

9. 3-bit Even Parity Generator (using NAND gates)

Function: Generates parity bit P so that the total number of 1s in A B C P is even. P = A ⊕ B ⊕ C. Each XOR is built from 4 NAND gates (8 NAND gates in total).

A	B	C	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

10. 4-bit Even Parity Generator (using NAND gates)

Function: Generates parity bit P so that the total number of 1s in A B C D P is even. P = A ⊕ B ⊕ C ⊕ D (3 XOR stages = 12 NAND gates).

A	B	C	D	P
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

11. AND gate using NAND gates

Function: Y = A·B. Implementation: NAND the inputs, then invert with a second NAND (inputs tied together): Y = ((A·B)')' – 2 NAND gates.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

12. AND gate using NOR gates

Function: Y = A·B. Implementation: invert each input with a NOR (A' = A↓A, B' = B↓B), then Y = A'↓B' = (A'+B')' = A·B – 3 NOR gates.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

13. D Flip-Flop using NAND gates

Function: Stores one bit. When the clock is enabled (CLK = 1) the output follows D: Q_{n+1} = D. When CLK = 0 the previous state is held. Built from 4 gating/steering NANDs + cross-coupled NAND latch.

CLK	D	Q_{n+1}	Q'_{n+1}
0	X	Q_n	Q'_n
1	0	0	1
1	1	1	0

14. Excess-3 code conversion from BCD

Function: Converts a BCD digit B3B2B1B0 into Excess-3 code (BCD + 3). Inputs 10–15 are invalid (don't care). E3 = B3 + B2B1 + B2B0; E2 = B2'B1 + B2'B0 + B2B1'B0'; E1 = B1 XNOR B0; E0 = B0'.

B3	B2	B1	B0	E3	E2	E1	E0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

15. Full Adder

Function: Adds three bits A, B and carry-in Cin. Sum = A ⊕ B ⊕ Cin; Cout = AB + Cin(A ⊕ B).

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

16. Full Subtractor

Function: Computes A − B − Bin. Difference = A ⊕ B ⊕ Bin; Borrow-out Bout = A'B + A'Bin + B·Bin.

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

17. Half Adder

Function: Adds two bits. Sum = A ⊕ B, Carry = A·B (1 XOR + 1 AND).

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

18. Half Subtractor

Function: Computes A − B. Difference = A ⊕ B, Borrow = A'·B (1 XOR + 1 NOT + 1 AND).

A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

19. Three-variable Boolean expression using basic gates

Function: Implements F(A,B,C) = AB + A'C using NOT, AND and OR gates (1 NOT, 2 AND, 1 OR). *Example expression used – replace with the one given in your lab manual.*

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

20. Three-variable Boolean expression using NAND gates

Function: Same F = AB + A'C in NAND-only form: F = ((AB)'·(A'C)')', with A' = NAND(A,A) (4 NAND gates). Truth table is identical to the basic-gate version.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

21. JK Flip-Flop using NAND gates

Function: Clocked flip-flop (table valid for CLK = 1; CLK = 0 holds state). J=K=0: no change; J=0,K=1: reset; J=1,K=0: set; J=K=1: toggle. $Q_{n+1} = J \cdot Q_n' + K' \cdot Q_n$.

J	K	Q_n	Q_{n+1}	State
0	0	0	0	No change
0	0	1	1	No change
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	1	Toggle
1	1	1	0	Toggle

22. Hexadecimal to Binary Encoder (16 : 4)

Function: Encodes the one active hex input line (0–F) into a 4-bit binary code. A3 = D8+…+DF; A2 = D4–D7 + DC–DF; A1 = D2,D3,D6,D7,DA,DB,DE,DF; A0 = D1,D3,D5,D7,D9,DB,DD,DF.

Active input	A3	A2	A1	A0
D0	0	0	0	0
D1	0	0	0	1
D2	0	0	1	0
D3	0	0	1	1
D4	0	1	0	0
D5	0	1	0	1
D6	0	1	1	0
D7	0	1	1	1
D8	1	0	0	0
D9	1	0	0	1
DA	1	0	1	0
DB	1	0	1	1
DC	1	1	0	0
DD	1	1	0	1
DE	1	1	1	0
DF	1	1	1	1

23. NAND gate using NOR gates

Function: Y = (A·B)'. Implementation: A' = A↓A, B' = B↓B; AND = A'↓B'; then invert with a NOR (tied inputs) – 5 NOR gates.

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

24. NOR gate using NAND gates

Function: Y = (A+B)'. Implementation: A' = NAND(A,A), B' = NAND(B,B); OR = NAND(A',B'); invert with a NAND (tied inputs) – 5 NAND gates.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

25. OR gate using NAND gates

Function: Y = A + B. Implementation: invert each input with a NAND, then NAND the results: Y = (A'·B')' = A + B – 3 NAND gates.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

26. OR gate using NOR gates

Function: Y = A + B. Implementation: NOR(A,B) followed by a NOR used as inverter – 2 NOR gates.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

27. SR Flip-Flop using NAND gates

Function: Clocked SR flip-flop (table valid for CLK = 1; CLK = 0 holds state). S=R=0: no change; S=0,R=1: reset; S=1,R=0: set; S=R=1 is forbidden. Q_{n+1} = S + R'·Q_n with S·R = 0.

S	R	Q_n	Q_{n+1}	State
0	0	0	0	No change
0	0	1	1	No change
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	X	Invalid
1	1	1	X	Invalid

28. SR Flip-Flop using NOR gates

Function: Clocked SR flip-flop with AND gating and cross-coupled NOR latch (table valid for CLK = 1). Same characteristic table as the NAND version; S=R=1 forces both outputs to 0 and is forbidden.

S	R	Q_n	Q_{n+1}	State
0	0	0	0	No change
0	0	1	1	No change
0	1	0	0	Reset
0	1	1	0	Reset
1	0	0	1	Set
1	0	1	1	Set
1	1	0	X	Invalid
1	1	1	X	Invalid

29. SR Latch using NAND gates

Function: Un-clocked latch from two cross-coupled NAND gates with **active-LOW** inputs S' and R'. S'=0 sets Q=1; R'=0 resets Q=0; both HIGH hold the state; both LOW is forbidden (Q = Q' = 1).

S'	R'	Q	Q'	State
0	0	1	1	Invalid
0	1	1	0	Set
1	0	0	1	Reset
1	1	Q_n	Q'_n	No change

30. SR Latch using NOR gates

Function: Un-clocked latch from two cross-coupled NOR gates with **active-HIGH** inputs. S=1 sets Q=1; R=1 resets Q=0; both LOW hold the state; both HIGH is forbidden (Q = Q' = 0).

S	R	Q	Q'	State
0	0	Q_n	Q'_n	No change
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Invalid

31. Seven-segment display (3-bit input)

Function: Decodes a 3-bit binary input (digits 0–7) into the signals driving segments a–g of a **common-cathode** display (1 = segment ON). For common-anode, invert all outputs.

A	B	C	Digit	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	1	1	0	1	1	0	0	0	0
0	1	0	2	1	1	0	1	1	0	1
0	1	1	3	1	1	1	1	0	0	1
1	0	0	4	0	1	1	0	0	1	1
1	0	1	5	1	0	1	1	0	1	1
1	1	0	6	1	0	1	1	1	1	1
1	1	1	7	1	1	1	0	0	0	0

32. XNOR gate using NAND gates

Function: Y = A ⊙ B = AB + A'B' (A ⊕ B inverted). Build XOR with 4 NANDs (N = NAND(A,B); Y' = NAND(NAND(A,N), NAND(B,N))), then invert with a 5th NAND.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

33. XNOR gate using NOR gates

Function: Y = AB + A'B'. N = NOR(A,B); NOR(A,N) = A'B; NOR(B,N) = AB'; Y = NOR(A'B, AB') – 4 NOR gates.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

34. XOR gate using NAND gates

Function: Y = A ⊕ B = AB' + A'B. N = NAND(A,B); Y = NAND(NAND(A,N), NAND(B,N)) – 4 NAND gates.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

35. XOR gate using NOR gates

Function: Y = A ⊕ B. Build XNOR with 4 NOR gates (N = NOR(A,B); XNOR = NOR(NOR(A,N), NOR(B,N))), then invert with a 5th NOR – 5 NOR gates.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

36. AND gate

Function: Y = A·B. Output is 1 only when all inputs are 1.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

37. OR gate

Function: Y = A + B. Output is 1 when at least one input is 1.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

38. NOT gate

Function: Y = A'. Inverts the input (single-input gate).

A	Y
0	1
1	0

39. XOR gate

Function: Y = A ⊕ B = A'B + AB'. Output is 1 when the inputs differ.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

40. XNOR gate

Function: Y = (A ⊕ B)' = AB + A'B'. Output is 1 when the inputs are equal.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

41. NOR gate

Function: Y = (A + B)'. Output is 1 only when all inputs are 0.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

42. NAND gate

Function: Y = (A·B)'. Output is 0 only when all inputs are 1.

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

43. T Flip-Flop

Function: Toggle flip-flop (table valid for CLK = 1). T=0: no change; T=1: output toggles. $Q_{n+1} = T \oplus Q_n$.

T	Q_n	Q_{n+1}	State
0	0	0	No change
0	1	1	No change
1	0	1	Toggle
1	1	0	Toggle

44. Three-variable Boolean expression using NOR gates

Function: Same F = AB + A'C = (A'+B)(A+C) in NOR-only form: F = NOR(NOR(A',B), NOR(A,C)), with A' = NOR(A,A) (4 NOR gates). Truth table identical to the basic-gate version.

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

45. Four-variable Boolean expression using basic gates

Function: Implements F(A,B,C,D) = AB + C'D using NOT, AND and OR gates (1 NOT, 2 AND, 1 OR). *Example expression used – replace with the one given in your lab manual.*

A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

46. Four-variable Boolean expression using NAND gates

Function: Same F = AB + C'D in NAND-only form: F = ((AB)'·(C'D)')', with C' = NAND(C,C) (4 NAND gates). Truth table identical to the basic-gate version.

A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

47. NOT gate using NAND gate

Function: Y = (A·A)' = A'. Tie both NAND inputs together – 1 NAND gate.

A	Y
0	1
1	0

48. NOT gate using NOR gate

Function: Y = (A+A)' = A'. Tie both NOR inputs together – 1 NOR gate.

A	Y
0	1
1	0

49. Two-variable Boolean expression using basic gates

Function: Implements F(A,B) = A + B' using one NOT and one OR gate. *Example expression used – replace with the one given in your lab manual.*

A	B	F
0	0	1
0	1	0
1	0	1
1	1	1

50. Two-variable Boolean expression using NAND gates

Function: Same F = A + B' = (A'·B)' in NAND-only form: F = NAND(A', B) with A' = NAND(A,A) (2 NAND gates). Truth table identical to the basic-gate version.

A	B	F
0	0	1
0	1	0
1	0	1
1	1	1